

Aula 40 - Apresentação de Arquitetura de Projeto

Descrição: Apresentação de uma arquitetura de projeto de ferramentaria na indústria 4.0 como exemplo de arquitetura de software para ser utilizado no projeto de SA.

Roteiro da Apresentação: Este material foi estruturado para guiar a apresentação em sala de aula sobre a arquitetura e engenharia do Sistema de Gestão de Ferramentaria (SGF). O foco é demonstrar as decisões técnicas, justificar as tecnologias adotadas e servir como um modelo de excelência em desenvolvimento de software.

1. Visão Geral e Objetivos do Projeto

O SGF foi projetado pela **AuraTech Development** para a **NexaForge Industrial Solutions** com um propósito claro, otimizar o chão de fábrica. É fundamental entender que um software não é apenas código, mas uma solução para problemas reais de negócio:

- **Mitigação de Perdas:** Evitar o extravio de ativos valiosos.
- **Redução de Setup:** Agilizar a preparação de máquinas para produção.
- **Manutenção Preventiva:** Garantir que as ferramentas estejam sempre calibradas e prontas para uso.
- **Automação:** Digitalizar e controlar o fluxo de empréstimos de forma temporal e rastreável.

2. Regras de Negócio e Segurança Corporativa

Para transformar a realidade industrial em lógica de programação, o sistema implementa regras estritas de segurança e rastreabilidade.

2.1. Controle de Acesso (RBAC)

O acesso é moldado pelo princípio do menor privilégio, dividido em papéis (Role-Based Access Control):

- **Administrador:** Acesso irrestrito, relatórios financeiros e baixas definitivas.
- **Ferramenteiro:** Operador do balcão, cadastra ativos e gerencia entradas/saídas.
- **Operador:** Técnico de fábrica, apenas consulta catálogo e histórico pessoal.

2.2. O Paradigma do "Soft Delete" (Baixa Lógica)

Ponto crucial, em sistemas industriais e de auditoria, **dados não se apagam**. É estritamente proibida a exclusão física (Delete). Ativos danificados sofrem uma "baixa lógica" com mudança de status e exigência de justificativa.

2.3. Controle Temporal (Datetime) e Punição Sistêmica

A temporalidade exige precisão. O sistema registra a saída, previsão e devolução com base no relógio do banco de dados. Se a devolução atrasar, o sistema bloqueia automaticamente novos empréstimos ao operador inadimplente, forçando a regularização.

3. Arquitetura de Software: O Padrão MVC e Isolamento

O sistema eleva o nível técnico ao afastar-se do "código espaguete" e abraçar o padrão Model-View-Controller (MVC) nativo, trazendo escalabilidade e manutenção reduzida.

Camada MVC	Responsabilidade no Sistema (SGF)
Model (Modelo)	Abstração de dados, comunicação com o MySQL e encapsulamento das regras de negócio pesadas.
View (Visão)	Páginas HTML5/CSS3 renderizadas para os usuários, livres de consultas ao banco de dados.
Controller (Controlador)	Cérebro do fluxo, recebe as requisições (via index.php), processa com o Model e devolve a View adequada.

Segurança via Front Controller

Apenas a pasta public/ fica exposta ao servidor web. O núcleo da aplicação (regras e dados) repousa protegido na

pasta app/, blindando o software contra injeções maliciosas. O roteamento utiliza namespaces via norma PSR-4.

4. Stack Tecnológico e Justificativas de Mercado

Neste tópico, vocês devem compreender **o porquê** das escolhas tecnológicas. A stack é enxuta, robusta e independente.

- **PHP 8.x Nativo:** Utilizado sem frameworks externos (como Laravel) para garantir imunidade a licenças e quebras de atualização de terceiros.
- **MySQL (ACID):** Banco de dados relacional escolhido pela garantia de transações seguras e suporte a grandes fluxos industriais.
- **JavaScript (Vanilla/ES6):** Essencial para requisições assíncronas (AJAX/Fetch) no Dashboard, atualizando KPIs de fábrica a cada 30 segundos sem recarregar a tela.

5. Qualidade de Código: Princípios SOLID

Destacamos a adoção de princípios avançados de Orientação a Objetos, que devem ser perseguidos por vocês:

- **SRP (Responsabilidade Única):** Modelos acessam dados, Controladores gerenciam fluxo, Visões mostram interface. Nada se mistura.
- **OCP (Aberto/Fechado):** Sistema projetado para receber novos canais de notificação (Ex: SMS, E-mail) futuramente, estendendo a lógica sem alterar o núcleo atual.
- **DIP (Inversão de Dependência):** Controladores não criam a conexão com o banco; eles recebem estaticamente a conexão PDO unificada.

6. Transações ACID (Exemplo Prático)

Como demonstrar estabilidade crítica? Ao registrar um empréstimo o sistema precisa, 1º Gravar o registro e 2º Atualizar o status da ferramenta. Se a energia cair entre os dois passos, os dados corrompem. A solução é o uso de transações PDO:

```
try {
    $this->db->beginTransaction();
    // 1. Inserir Empréstimo
    // 2. Atualizar Status para EMPRESTADA
    $this->db->commit();
} catch (Exception $e) {
    $this->db->rollBack(); // Desfaz tudo se houver erro!
}
```

Isso garante a "**Atomicidade**", um conceito vital para engenheiros de software.

7. Limitações de Escopo (Como gerenciar projetos)

Parte de ser um bom engenheiro de software é saber delimitar o escopo. Partes do projeto foram deixadas para o futuro de forma estratégica:

- **Integração com ERP (SAP/TOTVS):** A arquitetura está pronta para virar API depois, mas a V1 é autônoma.
- **Aplicativos Mobile Nativos:** Substituídos por interfaces web responsivas (HTML5) super leves para tablets industriais.

8. Conclusão

Este projeto serve como um **Laboratório Perfeito**. Ele ensina que desenvolver software corporativo vai muito além de "fazer funcionar". Envolve planejar a segurança (Front Controller), garantir integridade de dados (Transações/Soft Delete), facilitar manutenções futuras (SOLID/MVC) e entender as restrições e necessidades vitais de um chão de fábrica operando em ritmo de Indústria 4.0.